

Alur Pipeline Evaluasi Data Kode (Versi Paper)

Dokumen ini menjelaskan cara kerja pipeline evaluasi dalam format naratif untuk kebutuhan paper, fokus pada aliran data dari awal sampai akhir.

1. Sumber Data dan Field yang Diambil

Pipeline membaca data dari tabel code embeddings secara bertahap (batch), bukan sekaligus, supaya proses stabil untuk data besar.

Field utama yang dipakai per entri:

- id: identitas unik data
- prompt: instruksi atau pertanyaan awal
- generated_code: kode jawaban yang akan dievaluasi
- created_at: waktu data dibuat
- metadata: informasi tambahan jika tersedia

Catatan implementasi:

- Jika skema basis data memakai nama kolom code, sistem mengalias kolom tersebut menjadi generated_code saat proses evaluasi.

2. Tahap Pemrosesan per Entri

Setiap entri yang masuk diproses melalui beberapa tahap berikut.

1. Validasi dan analisis struktur kode

- Sistem mengidentifikasi bahasa pemrograman dan menghitung indikator kualitas statis, misalnya validitas sintaks, kompleksitas, serta ukuran kode.
- Metode yang digunakan:
 - AST (Abstract Syntax Tree) parsing untuk kode Python, untuk mengecek struktur sintaks dan menghitung elemen program secara lebih akurat.
 - Metrik kompleksitas dan maintainability menggunakan pendekatan static code metrics (cyclomatic complexity dan maintainability index).
 - Untuk non-Python, digunakan pendekatan heuristik berbasis pola (regex) dan pengecekan keseimbangan delimiter.

Contoh snippet (AST + static metrics):

```
import ast
from radon.complexity import cc_visit
from radon.metrics import mi_visit

tree = ast.parse(code)
function_count = sum(isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)) for
node in ast.walk(tree))
```

```

loop_count = sum(isinstance(node, (ast.For, ast.AsyncFor, ast.While)) for node in
ast.walk(tree))

complexities = cc_visit(code)
cyclomatic_complexity = float(max((block.complexity for block in complexities),
default=1.0))
maintainability_index = float(mi_visit(code, multi=True))

```

2. Pengukuran kesesuaian semantik

- Sistem mengukur seberapa selaras isi kode terhadap prompt menggunakan embedding similarity.
- Metode yang digunakan:
 - Embedding-based semantic similarity, yaitu representasi vektor untuk prompt dan kode lalu dihitung tingkat kedekatannya.

Contoh snippet (semantic similarity):

```

semantic_similarity = embedding_model.similarity(prompt, generated_code) if
generated_code.strip() else 0.0

```

3. Penilaian kualitas oleh model penilai

- Sistem menghasilkan skor kualitas konseptual (misalnya keselarasan jawaban, logika, keterbacaan, dan kelengkapan) untuk melengkapi hasil analisis statis.
- Metode yang digunakan:
 - LLM-as-a-Judge untuk memberi skor alignment, logic, quality, readability, dan completeness.
 - Jika layanan LLM tidak tersedia, sistem memakai heuristic scoring sebagai fallback agar pipeline tetap berjalan.

Contoh snippet (LLM-as-a-Judge):

```

judge_result = llm_judge.judge(
    prompt=prompt,
    code=generated_code,
    language=static_result.detected_language,
    semantic_similarity=semantic_similarity,
    static_result=static_result,
)

```

4. Perhitungan skor akhir

- Seluruh skor digabungkan menjadi satu skor akhir pada rentang 0 sampai 10.

Contoh snippet (rumus skor akhir):

```

final_score = round(
    max(

```

```

0.0,
min(
    0.40 * alignment
    + 0.25 * logic
    + 0.20 * (similarity * 10.0)
    + 0.10 * quality
    + 0.05 * static_score,
    10.0,
),
),
2,
)

```

5. Deteksi duplikasi

- Sistem membandingkan fingerprint konten prompt dan kode.
- Jika ada konten identik, salinan pertama dipertahankan, sisanya ditandai sebagai duplikat.
- Metode yang digunakan:
 - Hashing SHA-256 pada gabungan prompt dan generated code untuk identifikasi duplikasi persis (exact duplicate).

Contoh snippet (duplicate hashing):

```

from hashlib import sha256

content_hash = sha256(f"{prompt.strip()}\n---\n{generated_code.strip()}".encode("utf-8")).hexdigest()
duplicate_of = seen_hashes.get(content_hash)
if duplicate_of is None:
    seen_hashes[content_hash] = str(row["id"])

```

3. Analisis Anomali

Setelah semua entri dalam satu siklus diproses, pipeline menjalankan deteksi anomali berbasis fitur numerik (misalnya kompleksitas, similarity, dan skor).

Metode yang digunakan:

- Isolation Forest dengan kontaminasi 10% untuk menandai data yang menyimpang dari pola mayoritas.
- Fitur utama yang dianalisis mencakup panjang kode, kompleksitas, semantic similarity, static score, dan skor akhir sebelum anomali.

Contoh snippet (Isolation Forest):

```

import numpy as np
from sklearn.ensemble import IsolationForest

matrix = np.array(
    [

```

```

    [
        row["line_count"],
        row["cyclomatic_complexity"],
        row["semantic_similarity"],
        row["static_score"],
        row["final_score_pre_anomaly"],
    ]
    for row in feature_rows
],
dtype=float,
)

model = IsolationForest(contamination=0.1, random_state=42)
labels = model.fit_predict(matrix) # -1: anomaly, 1: normal

```

Fungsi tahap ini:

- Menandai entri yang mencurigakan sebagai suspicious
- Menambah konteks pada laporan kualitas

Catatan:

- Sinyal anomali bersifat advisory untuk monitoring; tidak otomatis menghapus data.

4. Pengambilan Keputusan (Filtering)

Setelah skor tersedia, setiap entri masuk ke salah satu kategori berikut.

1. DUPLICATE

- Entri terdeteksi duplikat dari entri sebelumnya.

2. INVALID

- Kesesuaian semantik berada di bawah ambang minimal.

3. LOW_QUALITY

- Skor akhir berada di bawah ambang mutu minimum.

4. REVIEW_REQUIRED

- Skor masuk area abu-abu: belum cukup buruk untuk dihapus, tapi belum cukup baik untuk langsung dipertahankan.

5. VALID

- Lolos seluruh kriteria kualitas.

Secara operasional:

- DUPLICATE, INVALID, LOW_QUALITY masuk kandidat penghapusan
- REVIEW_REQUIRED dipertahankan untuk peninjauan manual

- VALID dipertahankan sebagai data bersih

Contoh snippet (aturan keputusan):

```
if duplicate_of:
    deletion_reason = "DUPLICATE"
elif semantic_similarity < semantic_similarity_threshold:
    deletion_reason = "INVALID"
elif final_score < review_cutoff:
    deletion_reason = "LOW_QUALITY"
elif final_score < final_score_threshold:
    deletion_reason = "REVIEW_REQUIRED"
else:
    deletion_reason = None # VALID
```

5. Mekanisme Keamanan Sebelum Hapus Data

Sebelum penghapusan dilakukan, sistem selalu membuat berkas backup berisi kandidat data yang akan dihapus.

Tujuan:

- Menjaga audit trail
- Memungkinkan rollback jika dibutuhkan

Jika mode dry run aktif:

- Pipeline hanya simulasi (tanpa delete fisik)
- Tetap menghasilkan ringkasan hasil evaluasi

6. Output Akhir Pipeline

Dalam satu siklus evaluasi, pipeline menghasilkan tiga keluaran utama:

1. Backup kandidat penghapusan
 - Berisi salinan data yang masuk kategori hapus
2. Laporan evaluasi per siklus
 - Berisi statistik agregat, termasuk jumlah data valid, review, dihapus, rata-rata skor, dan jumlah entri suspicious
3. Ringkasan statistik terbaru
 - Snapshot status terkini untuk monitoring cepat

7. Ringkasan Alur End-to-End

Secara ringkas, alurnya adalah:

Pengambilan data dari database -> evaluasi kualitas per entri -> deteksi anomali -> klasifikasi kualitas -> backup kandidat delete -> penghapusan terkontrol (opsional saat bukan dry run) -> pembuatan laporan.

Dengan alur ini, pipeline berperan sebagai quality control otomatis untuk menjaga kualitas data kode sekaligus tetap aman secara operasional karena ada backup dan jejak evaluasi.

8. Hasil Report Evaluasi (Contoh Run Nyata)

Contoh hasil run diambil dari file laporan evaluasi terbaru pada repository ini.

Identitas run:

- run_id: 20260312T094424Z
- trigger: manual-background
- durasi: 1087.13 detik (sekitar 18 menit)
- total data diproses: 678 entri

Ringkasan hasil:

- valid_entries: 647 (95.43%)
- review_entries: 1 (0.15%)
- deleted_entries: 30 kandidat (4.42%)
- suspicious_entries: 67 (9.88%)
- average_similarity: 0.8774
- average_score: 8.6149

Rincian kategori kandidat delete:

- DUPLICATE: 23 entri (76.67% dari kandidat delete)
- LOW_QUALITY: 7 entri (23.33% dari kandidat delete)
- INVALID: 0 entri pada run ini

Konfigurasi ambang yang digunakan:

- semantic_similarity_threshold: 0.8
- review_score_threshold: 4.8
- final_score_threshold: 5.2

Catatan penting interpretasi:

- Nilai dry_run pada report ini adalah true.
- Artinya, angka deleted_entries menunjukkan jumlah kandidat hapus berdasarkan aturan kualitas, bukan penghapusan fisik ke database.